Name - Sanket Akkolkar

Roll_NO - 25

Division - AIML(A)

Logistic Regression for Binary Classification:

Objective

To implement Logistic Regression for a binary classification problem (Placed / Not Placed) using Study_Hours and Previous_Year_Marks as input features.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, auc
```

Step 2: Load Dataset

```
# Load the Passed dataset from the specified file path into a Pandas DataFrame
df = pd.read_csv("/content/drive/MyDrive/Student_result.csv")
```

```
# Display the first five rows of the DataFrame to quickly inspect the data
df.head()
```

	Study_Hours	Previous_Year_Marks	Passed
0	1.0	35	0
1	2.0	40	0
2	2.5	45	0
3	3.0	48	0
4	3.5	50	0

```
df.head()
```

	Study_Hours	Previous_Year_Marks	Passed
0	1.0	35	0
1	2.0	40	0
2	2.5	45	0
3	3.0	48	0
4	3.5	50	0

```
# Display concise information about the DataFrame such as column names, data types, and non-null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
```

```
 0   Study_Hours          100 non-null    float64
 1   Previous_Year_Marks  100 non-null    int64
 2   Passed               100 non-null    int64
dtypes: float64(1), int64(2)
memory usage: 2.5 KB
```

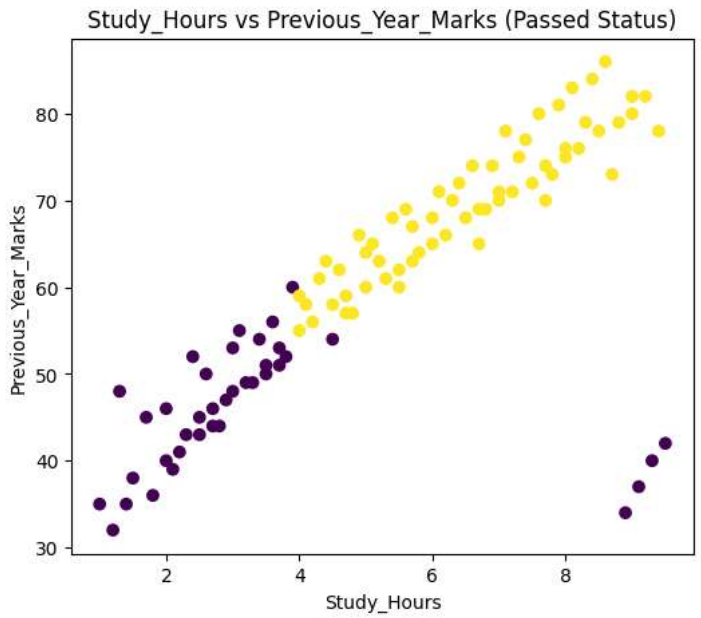
```
# Check for missing (null) values in each column of the DataFrame
df.isnull().sum()
```

	0
Study_Hours	0
Previous_Year_Marks	0
Passed	0

dtype: int64

```
# Create a scatter plot of Study_Hours vs Previous_Year_Marks
# Color points based on Passed status (0 = Not Placed, 1 = Placed)

plt.figure(figsize=(6, 5))
plt.scatter(df['Study_Hours'], df['Previous_Year_Marks'], c=df['Passed'])
plt.xlabel('Study_Hours')
plt.ylabel('Previous_Year_Marks')
plt.title('Study_Hours vs Previous_Year_Marks (Passed Status)')
plt.show()
```



Step 4: Feature Selection

```
# Select the independent variables (features): Study_Hours and Previous_Year_Marks
X = df[['Study_Hours', 'Previous_Year_Marks']]

# Select the dependent variable (target): Passed status (1 = Placed, 0 = Not Placed)
y = df['Passed']
```

Step 5: Train-Test Split

```
# Split the dataset into training and testing sets
# 80% data is used for training and 20% for testing
# stratify=y ensures both classes are equally represented in train and test sets
# random_state ensures the same split every time the code is executed
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Step 6: Feature Scaling

```
# Create a StandardScaler object to normalize the feature values
scaler = StandardScaler()

# Fit the scaler on training data and transform it
# This standardizes features to have mean = 0 and standard deviation = 1
X_train = scaler.fit_transform(X_train)

# Transform the test data using the same scaler
# (Important: do NOT fit again on test data)
X_test = scaler.transform(X_test)
```

Step 7: Model Training

```
# Create an object of the Logistic Regression model
model = LogisticRegression()
```

```
# Train (fit) the model using the training data
# The model learns the relationship between Study_Hours, Previous_Year_Marks and Passed outcome
model.fit(X_train, y_train)
```

```
LogisticRegression()
```

Step 8: Predictions

```
# Predict the Passed outcome for the test dataset using the trained model
y_pred = model.predict(X_test)
```

Step 9: Model Evaluation

```
# Display the accuracy of the model
# Accuracy shows the percentage of correctly classified instances
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

```
# Display the confusion matrix
# It shows counts of True Positives, True Negatives, False Positives, and False Negatives
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
 [[ 8  0]
 [ 0 12]]
```

```
# Display the classification report
# It includes precision, recall, F1-score, and support for each class
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         8
           1       1.00      1.00      1.00        12

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20
```

Step 10: ROC Curve

```
# Predict the probability of the positive class (Placed = 1) for the test data
# [:, 1] selects the probability of class '1'
y_prob = model.predict_proba(X_test)[:, 1]

# Compute False Positive Rate (FPR) and True Positive Rate (TPR) for different thresholds
fpr, tpr, _ = roc_curve(y_test, y_prob)

# Calculate Area Under the Curve (AUC)
roc_auc = auc(fpr, tpr)

# Plot the ROC curve
plt.plot(fpr, tpr, label='AUC = %.2f' % roc_auc)

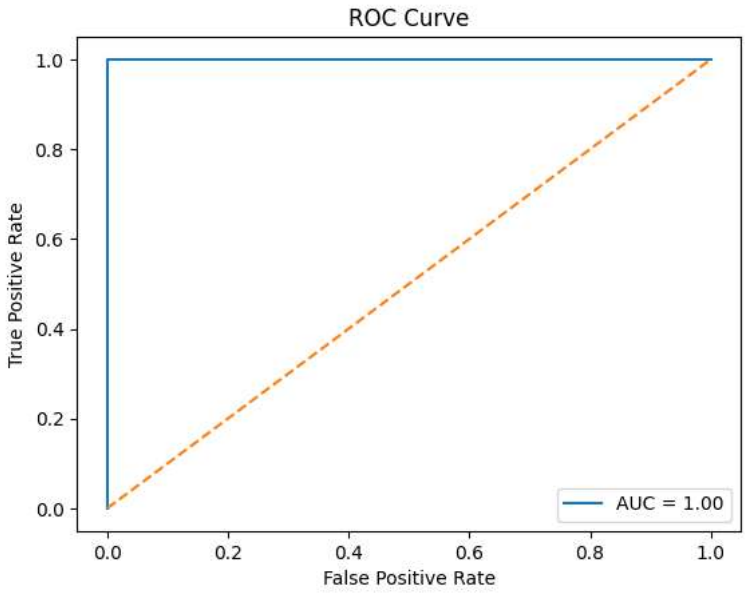
# Plot the diagonal reference line (random classifier)
plt.plot([0, 1], [0, 1], linestyle='--')

# Label the axes
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

# Set the title of the plot
plt.title('ROC Curve')

# Display legend
plt.legend()

# Show the plot
plt.show()
```



Step 11: Prediction for New Data

```
# Create a new data point for a student with Study_Hours = 7.8 and Previous_Year_Marks = 120
new_student = np.array([[10, 70]])

# Apply the same scaling used for training data
new_student_scaled = scaler.transform(new_student)

# Predict the Passed outcome for the new student
prediction = model.predict(new_student_scaled)

# Display the prediction result in a readable format
print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
```

```
Prediction: Pass
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
```

```
  warnings.warn(
```

```
# Take input values from the user
Study_Hours = float(input("Enter Study_Hours: "))
Previous_Year_Marks = float(input("Enter Previous_Year_Marks: "))

# Create input array in required 2D format
new_student = np.array([[Study_Hours, Previous_Year_Marks]])

# Apply the same scaling used during model training
new_student_scaled = scaler.transform(new_student)

# Predict the Passed outcome for the given input
prediction = model.predict(new_student_scaled)

# Display the prediction result in a user-friendly format
print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
```

```
Enter Study_Hours: 34
Enter Previous_Year_Marks: 90
Prediction: Pass
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

```
import pickle
```

```
pickle.dump(model,open('BCModel.pkl','wb'))
```

Flask Code

save this file as 'app.py'

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

# Load Trained Logistic Regression Model
model = pickle.load(open("BCModel.pkl", "rb"))


@app.route('/')
def home():
    return render_template("index.html")


@app.route('/predict', methods=['POST'])
def predict():

    # Read user inputs
    Study_Hours = float(request.form['Study_Hours'])
    Previous_Year_Marks = float(request.form['Previous_Year_Marks'])

    # Prediction
    prediction = model.predict(np.array([[Study_Hours, Previous_Year_Marks]]))

    # Convert numeric prediction to text
    if prediction[0] == 1:
        result = "Student is Pass"
    else:
        result = "Student is Fail"

    return render_template(
        "index.html",
        prediction_text=result
    )


if __name__ == "__main__":
    app.run(debug=True)
```

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inst
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

HTML Code

Save this file as 'index.html'

```
<!DOCTYPE html>
<html lang="en">

<head>

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Passed Prediction</title>

<style>

*{
    margin:0;
    padding:0;
    box-sizing:border-box;
    font-family:Arial, Helvetica, sans-serif;
}

body{

    background:#eef4ff;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;

}

.container{

    width:430px;
    background:white;
    padding:35px;
    border-radius:15px;
    box-shadow:0px 8px 20px rgba(0,0,0,.2);

}

h1{

    text-align:center;
    color:#1565c0;

}

h3{

    text-align:center;
    color:#666;
    margin-top:8px;
    margin-bottom:25px;

}

label{

    display:block;
    font-weight:bold;
    margin-top:15px;
    margin-bottom:6px;

}
```

```
input{

    width:100%;
    padding:12px;
    border:1px solid #ccc;
    border-radius:8px;
    font-size:16px;

}

button{

    width:100%;
    padding:13px;
    margin-top:25px;
    background:#1565c0;
    color:white;
    border:none;
    border-radius:8px;
    font-size:18px;
    cursor:pointer;
    transition:.3s;

}

button:hover{

    background:#0d47a1;

}

.result{

    margin-top:25px;
    padding:18px;
    border-radius:8px;
    text-align:center;
    font-size:22px;
    font-weight:bold;
    background:#e8f5e9;
    color:green;

}

footer{

    margin-top:20px;
    text-align:center;
    color:#777;
    font-size:13px;

}

</style>

</head>

<body>

<div class="container">

<h1>Passed Prediction System</h1>

<h3>Binary Classification using Logistic Regression</h3>

<form action="/predict" method="POST">

<label>Enter Student Study_Hours</label>

<input
type="number"
name="Study_Hours"
step="0.01"
min="0"
max="10"
placeholder="Example : 10"
required>
```

```
<label>Enter Previous_Year_Marks Score</label>

<input
type="number"
name="Previous_Year_Marks"
placeholder="Example : 80"
required>

<button type="submit">

Predict Passed

</button>

</form>

{% if prediction_text %}

<div class="result">

{{ prediction_text }}

</div>

{% endif %}

<footer>

By Dr. Sachin Balawant Takmare

</footer>

</div>

</body>

</html>
```

```
  File "/tmp/ipykernel_3256/2657364888.py", line 26
    height:100vh;
           ^
SyntaxError: invalid decimal literal
```